

Softwarequalität

Hausarbeit

Studiengang Informatik

Duale Hochschule Baden-Württemberg Karlsruhe

Etienne Luke Josef Bader

Eingereicht am: 19.05.2026

Matrikelnummer, Kurs: 9578543, TINF23B2

Betreuer an der DHBW: Dennis Kube, Jonathan Schwarzenböck

Inhalt

1	Einleitung	1
1.1	Projektbeschreibung	1
1.2	Ziel der Arbeit	1
2	Grundlagen	3
2.1	Clean Architecture	3
2.1.1	Motivation und Ziel	3
2.1.2	Die Dependency Rule	4
2.1.3	Schichtenaufbau	4
2.1.4	Innere Schichten definieren Interfaces	5
2.2	Softwarequalität nach ISO/IEC 25010	5
3	Betrachtung des Projekts	7
3.1	Architekturaufbau des Projekts	7
3.2	Korrekte Umsetzung im Überblick	8
3.3	Strukturelle Abweichungen	9
3.3.1	Direkter Repository-Zugriff in der Plugin-Schicht	9
3.3.2	Framework-Abhängigkeit in der Application-Schicht	10
4	Analyse und Bewertung	11
4.1	Bewertung: Direkter Repository-Zugriff	11
4.1.1	Fehlende Kapselung der Anwendungslogik	11
4.1.2	Auswirkung auf Modularity	12
4.1.3	Auswirkung auf Modifiability	12
4.2	Bewertung: Framework-Abhängigkeit in der Application-Schicht	13
4.2.1	Spring als Compile-Abhängigkeit	13
4.2.2	Auswirkung auf Modularity und Modifiability	13
4.3	Risikobewertung mittels FMEA	14
4.4	Gesamtbewertung	15
5	Optimierungsmaßnahmen	17
5.1	Repository-Zugriffe in Controller eliminieren	17
5.2	Service-Annotation verschieben	17
5.3	Erwartete Auswirkungen im PDCA-Rahmen	17
6	Fazit	18

A Literatur	19
B Glossar	20



Einleitung

1.1 Projektbeschreibung

Das Sportwettbewerbs-Managementtool ist eine im Rahmen der Vorlesung Advanced Software Engineering an der Dualen Hochschule Baden-Württemberg Karlsruhe entwickelte Anwendung. Sie unterstützt Organisatorinnen und Organisatoren bei der Planung und Durchführung von Sportwettbewerben und bietet Funktionen zur Verwaltung von Personen, Teams, Wettkämpfen, Spielplänen, Lageplänen und Tickets. Das System besteht aus einem Java-Backend auf Basis des Spring-Boot-Frameworks sowie einem React-Frontend und wird über eine REST-API angesteuert.

Ein zentrales Merkmal des Projekts ist die bewusste Entscheidung für eine Clean Architecture, auch bekannt als Onion Architecture. Diese gliedert das System in vier klar voneinander getrennte Schichten: Domain, Application, Adapter und Plugins. Ziel dieser Architektur ist es, die fachliche Kernlogik von technischen Details wie Frameworks oder Persistenzmechanismen zu entkoppeln und so langfristige Wartbarkeit und Erweiterbarkeit sicherzustellen.

1.2 Ziel der Arbeit

Im Mittelpunkt dieser Hausarbeit steht die Frage, inwieweit die gewählte Architektur im Sinne des Softwarequalitätsmanagements tatsächlich konsequent umgesetzt wurde. Als Bewertungsmaßstab dienen zwei Teilmerkmale des Qualitätsmerkmals Maintainability aus dem internationalen Standard ISO/IEC 25010: Modularity und Modifiability. Modularity beschreibt, in welchem Maße ein System aus unabhängigen Komponenten besteht, sodass Änderungen an einer Komponente möglichst geringe Auswirkungen auf andere haben. Modifiability

beschreibt, wie effektiv das System verändert werden kann, ohne dabei unbeabsichtigte Seiteneffekte in anderen Teilen zu erzeugen.

Beide Teilmerkmale sind unmittelbar mit dem Anspruch der Clean Architecture verknüpft und eignen sich daher als konkreter, messbarer Maßstab zur Bewertung ihrer Umsetzung. Die Analyse konzentriert sich auf den `CompetitionController` als repräsentatives Fallbeispiel, an dem sich eine systematische Abweichung vom eigenen Architekturanspruch belegen lässt. Daraus werden anschließend begründete Optimierungsmaßnahmen abgeleitet.



Grundlagen

2.1 Clean Architecture

2.1.1 Motivation und Ziel

Moderne Softwaresysteme unterliegen einem beständigen Wandel. Technologien, Frameworks und Abhängigkeiten veralten, werden ersetzt oder weiterentwickelt – häufig in einem Rhythmus von wenigen Jahren [1, S. 2]. Eine Architektur, die eng an konkrete Technologien geknüpft ist, altert zwangsläufig mit diesen zusammen und erschwert spätere Anpassungen erheblich.

Die Clean Architecture begegnet diesem Problem durch eine klare strukturelle Trennung von langlebigem und kurzlebigem Code. Ihr zentrales Ziel ist es, einen technologieunabhängigen Kern zu schaffen, der sämtliche fachliche Logik enthält, und alle technischen Details – Datenbanken, Frameworks, Benutzeroberflächen – als austauschbare Randkomponenten zu behandeln [1, S. 9]. Die Metapher der Zwiebel (Onion Architecture) beschreibt diesen Aufbau treffend: Der Kern ist langlebig und stabil, die äußeren Schichten sind kurzlebiger und leichter ersetzbar [1, S. 9].

Das angestrebte Ergebnis ist ein System, in dem Technologieentscheidungen spät getroffen oder nachträglich revidiert werden können, ohne den Anwendungskern zu berühren [1, S. 8]. Robert C. Martin, der die Clean Architecture maßgeblich geprägt hat, fasst dies so zusammen: Eine gute Architektur maximiert die Anzahl der Entscheidungen, die noch nicht getroffen werden müssen [2, S. 141].

2.1.2 Die Dependency Rule

Das zentrale Prinzip der Clean Architecture ist die Dependency Rule. Sie besagt, dass Abhängigkeiten zwischen Systemteilen ausschließlich von außen nach innen zeigen dürfen [1, S. 11]. Dabei ist zwischen zwei Arten von Pfeilen zu unterscheiden: Abhängigkeitspfeile (Compile-Time Dependencies) zeigen an, welchen Code eine Klasse direkt referenziert und ohne den sie nicht kompilierbar wäre. Aufrufpfeile (Runtime Dependencies) beschreiben hingegen, welche Klasse zur Laufzeit welche andere aufruft — diese können in beide Richtungen verlaufen [1, S. 12].

Die Dependency Rule betrifft ausschließlich die Abhängigkeitspfeile: Innere Schichten dürfen äußere Schichten weder kennen noch referenzieren. Eine Verletzung dieser Regel — etwa wenn eine innere Schicht eine Klasse einer äußeren Schicht importiert — untergräbt die gesamte Architektur, da Änderungen an der äußeren Schicht dann unmittelbar Auswirkungen auf den eigentlich stabilen Kern haben [2, S. 203].

2.1.3 Schichtenaufbau

Die Clean Architecture gliedert ein System typischerweise in vier Schichten [1, S. 14]:

Die **Domain-Schicht** bildet den innersten Kern. Sie enthält die zentralen Geschäftsobjekte (Entities) sowie die organisationsweit gültigen Geschäftsregeln, die unabhängig davon existieren, ob sie in einer konkreten Anwendung nachmodelliert wurden [1, S. 18–19]. Diese Schicht sollte sich am seltensten ändern und ist vollständig immun gegen Änderungen an Infrastrukturdetails wie Anzeige, Transport oder Speicherung [1, S. 18].

Die **Application-Schicht** enthält die anwendungsspezifischen Anwendungsfälle (Use Cases). Sie steuert den Daten- und Aktionsfluss zwischen den Entities und implementiert Regeln, die nur für den konkreten Anwendungsfall gelten — beispielsweise Workflows oder Validierungsschritte [1, S. 23–24]. Diese Schicht ist isoliert von Änderungen an Datenbank oder Benutzeroberfläche, reagiert aber auf veränderte Anforderungen [1, S. 25].

Die **Adapter-Schicht** vermittelt zwischen der Anwendungslogik und der Außenwelt. Sie übernimmt Formatkonvertierungen, sodass Daten aus der Anwendungsschicht in ein für die Plugins geeignetes Format überführt werden und umgekehrt [1, S. 30]. Ihr Ziel ist die vollständige Entkopplung von innen und außen — kein SQL in der Anwendung selbst, keine Renderlogik im Kern [1, S. 31].

Die **Plugin-Schicht** ist die äußerste Schicht und enthält Frameworks, Datentransportmittel und andere technische Werkzeuge wie Datenbank, Web und Benutzeroberfläche [1, S. 37]. Diese Schicht soll hauptsächlich Delegationscode enthalten, der Aufrufe an die Adapter

weiterleitet. Auf keinen Fall darf sie Anwendungslogik enthalten — alle Entscheidungen sollen bereits in den inneren Schichten gefallen sein [1, S. 37–38].

Diese Schichtenstruktur spiegelt sich direkt in den erwarteten Lebenszyklen wider: Während Domain-Code Jahrzehnte Bestand haben kann, veralten Plugin-Code und Frameworks mitunter innerhalb von Wochen bis Monaten [1, S. 50].

2.1.4 Innere Schichten definieren Interfaces

Ein wesentliches Umsetzungsmittel der Dependency Rule ist die konsequente Nutzung von Interfaces: Innere Schichten definieren Schnittstellen, äußere Schichten implementieren diese [1, S. 16]. So kann die Domain-Schicht beispielsweise ein Repository-Interface definieren, ohne zu wissen, ob die konkrete Implementierung eine Datei, eine relationale Datenbank oder einen Webservice verwendet. Die Plugin-Schicht implementiert dieses Interface und koppelt sich damit an die innere Schicht — nicht umgekehrt. Dieses Muster wird auch als Dependency Inversion Principle bezeichnet und ist eine der tragenden Säulen der Clean Architecture [2, S. 91].

2.2 Softwarequalität nach ISO/IEC 25010

ISO/IEC 25010 ist der internationale Standard zur Beschreibung und Bewertung von Softwarequalität. Er definiert ein hierarchisches Qualitätsmodell, das Qualitätsmerkmale in Hauptmerkmale und Teilmerkmale untergliedert [3, Abschn. 4.2]. Als Bewertungsmaßstab für diese Arbeit sind zwei Teilmerkmale des Hauptmerkmals **Maintainability** (Wartbarkeit) relevant.

Modularity beschreibt den Grad, zu dem ein System aus voneinander unabhängigen Komponenten besteht, sodass eine Änderung an einer Komponente möglichst geringe Auswirkungen auf andere Komponenten hat [3, Abschn. 4.2.7.1]. Ein System mit hoher Modularität ermöglicht es, einzelne Teile isoliert zu verstehen, zu testen und auszutauschen, ohne das Gesamtsystem zu destabilisieren.

Modifiability beschreibt den Grad, zu dem ein System effektiv und effizient verändert werden kann, ohne dabei unbeabsichtigte Seiteneffekte in anderen Teilen des Systems zu erzeugen [3, Abschn. 4.2.7.3]. Dieses Teilmerkmal ist eng mit Modularität verknüpft: Ein System mit schlechter Modularität erschwert gezielte Modifikationen, da Änderungen an einer Stelle unkontrolliert auf andere Stellen ausstrahlen.

Beide Teilmerkmale sind unmittelbar mit dem Anspruch der Clean Architecture verknüpft. Die Dependency Rule und die daraus resultierende Schichtentrennung sind präzise darauf

ausgelegt, Modularität und Modifiability zu maximieren. Die Clean Architecture liefert damit nicht nur ein Designprinzip, sondern zugleich den Maßstab, an dem ihre eigene Umsetzung gemessen werden kann.

3

Betrachtung des Projekts

Dieses Kapitel beschreibt zunächst den Gesamtaufbau des Sportwettbewerbs- Management-tools in Bezug auf seine Architektur und bewertet anschließend, inwiefern die Clean Architecture im Projekt umgesetzt wurde. Dabei werden sowohl gelungene Aspekte als auch strukturelle Abweichungen vom Architekturanspruch herausgearbeitet, die als Grundlage für die Analyse in Kapitel 4 dienen.

3.1 Architekturaufbau des Projekts

Das Backend des Projekts ist als Maven-Multi-Modul-Projekt organisiert und folgt dem in Kapitel 2 beschriebenen Schichtenmodell der Clean Architecture. Die vier Module sind entsprechend ihrer Schicht benannt und nummeriert: `3_domain`, `2_application`, `1_adapter` und `0_plugins`. Diese Nummerierung spiegelt die Abhängigkeitsrichtung wider: Ein Modul mit niedrigerer Nummer darf nur von Modulen mit höherer Nummer abhängig sein, nicht umgekehrt. Die Modulstruktur selbst entspricht damit der Empfehlung von Briem, Schichten als separate Projekte umzusetzen, sodass der Compiler unzulässige Abhängigkeiten in die falsche Richtung verhindert [1, S. 44].

Die **Domain-Schicht** (`3_domain`) enthält die zentralen Geschäftsobjekte des Systems: `Competition`, `Match`, `Standings`, `Team`, `Person` und ihre Subtypen (`Athlete`, `Coach`, `Official`, `Visitor`), `Ticket` sowie `Siteplan`. Zu jedem dieser Aggregate existiert ein Repository-Interface, das in der Domain-Schicht definiert ist – etwa `CompetitionRepository` oder `TicketRepository`. Die Domain-Schicht enthält keinerlei Framework-Imports; insbesondere sind keine Spring-Annotierungen vorhanden. Dieses Vorgehen entspricht der Grundregel der

Clean Architecture, wonach der Anwendungs- und Domaincode frei von Abhängigkeiten gegenüber Frameworks sein soll [1, S. 16].

Die **Application-Schicht** (`2_application`) enthält für jeden fachlichen Bereich einen Service sowie die zugehörigen UseCase-Interfaces. Für den Bereich Wettkämpfe sind dies beispielsweise `CreateCompetitionUseCase` und `GetStandingsUseCase`, die von `CompetitionService` implementiert werden. Alle UseCases kommunizieren ausschließlich über Command-Objekte und DTOs mit den äußeren Schichten – eine Entkopplung, die verhindert, dass Domain-Objekte direkt nach außen exponiert werden.

Die **Adapter-Schicht** (`1_adapter`) enthält Mapper-Klassen, die zwischen den internen Domain-Objekten und den Dateitransfer-Objekten (DTOs) der Persistenzschicht übersetzen. So wandelt etwa `CompetitionFileMapper` ein `Competition-Domain-Objekt` in ein `CompetitionFileDto` um und umgekehrt. Diese Schicht hat keine Kenntnis von HTTP, REST oder Spring.

Die **Plugin-Schicht** (`0_plugins`) gliedert sich in zwei Teilmodule: `plugin_rest` enthält die REST-Controller auf Basis von Spring Boot, `plugin_io` die dateibasierten Repository-Implementierungen. Die Repository-Implementierungen – etwa `FileCompetitionRepository` – implementieren jeweils das in der Domain-Schicht definierte Repository-Interface und erfüllen damit das Dependency-Inversion-Prinzip korrekt: Die innere Schicht definiert die Schnittstelle, die äußere Schicht implementiert sie [1, S. 16].

3.2 Korrekte Umsetzung im Überblick

In weiten Teilen des Projekts ist die Clean Architecture konsequent umgesetzt. Die UseCase-Interfaces und ihre Implementierungen in der Application-Schicht sind vollständig vorhanden. Der `PersonController` etwa nutzt für alle vier Schreiboperationen (Anlegen von Athlete, Coach, Official, Visitor) ausschließlich die entsprechenden UseCase-Interfaces – `CreateAthleteUseCase`, `CreateCoachUseCase`, `CreateOfficialUseCase` und `CreateVisitorUseCase` – ohne direkt auf ein Repository zuzugreifen. Gleiches gilt für den `TicketController`, dessen beide schreibenden Endpunkte (`/create` und `/sellTicket`) vollständig über `CreateTicketUseCase` und `SellTicketUseCase` abgewickelt werden. Dieses Vorgehen entspricht der in Kapitel 2 beschriebenen Rolle der Plugin-Schicht: Sie enthält hauptsächlich Delegationscode, der Aufrufe an die inneren Schichten weiterleitet [1, S. 37].

Auch die Domain-Schicht selbst zeigt qualitativ hochwertige Umsetzungsbeispiele. Die Klasse `SportResultComparator` nutzt das Strategy-Pattern über das Interface `ScoringStrategy`, sodass neue Sportarten hinzugefügt werden können, ohne bestehenden Code zu verändern – ein direktes Anwendungsbeispiel des Open-Closed-Prinzips.

3.3 Strukturelle Abweichungen

Neben diesen gelungenen Aspekten finden sich im Projekt zwei strukturelle Abweichungen von der Clean Architecture, die für die Bewertung nach ISO/IEC 25010 relevant sind.

3.3.1 Direkter Repository-Zugriff in der Plugin-Schicht

Der `CompetitionController` hält als Instanzvariable nicht nur die UseCase-Interfaces `CreateCompetitionUseCase` und `GetStandingsUseCase`, sondern zusätzlich ein direktes Referenz auf `CompetitionRepository` — ein Interface, das in der Domain-Schicht definiert ist. Von den sieben Endpunkten des Controllers umgehen fünf die Application-Schicht vollständig und rufen das Repository direkt auf:

- GET-Methode `/getById`

→ `competitionRepository.findCompetitionById(id)`

- GET-Methode `/getMatchesByCompetitionId`

→ `competitionRepository.getAllMatchesFromCompetitionId(id)`

- GET-Methode `/getMatchById`

→ `competitionRepository.findMatchById(id)`

- GET-Methode `/getCompetitionByCompetitionName`

→ `competitionRepository.getCompetitionByName(name)`

- POST-Methode `/registerMatchResults`

→ `competitionRepository.registerResultsForMatchById(...)`

Lediglich zwei Endpunkte — GET `/getStandingsFromCompetition` und POST `/create` — nutzen den vorgesehenen Weg über die UseCase-Interfaces. Die Plugin-Schicht überspringt damit für den Großteil ihrer Operationen die Application-Schicht und greift direkt auf die Domain-Schicht zu. Dies ist eine Verletzung der Dependency Rule, die vorschreibt, dass die Plugin-Schicht grundsätzlich nur auf die Adapter-Schicht zugreift [1, S. 37]. Das Muster des direkten Repository-Zugriffs findet sich darüber hinaus auch im `PersonController` (ein Lesezugriff via `PersonRepository`) und im `TicketController` (ein Lesezugriff via `TicketRepository`), ist jedoch im `CompetitionController` am ausgeprägtesten.

3.3.2 Framework-Abhängigkeit in der Application-Schicht

Eine zweite Abweichung betrifft die Abhängigkeitsstruktur auf Modul-Ebene. Die `pom.xml` der Application-Schicht deklariert `spring-boot-starter-web` als Compile-Zeit-Abhängigkeit – nicht als Test-Abhängigkeit. Dies hat zur Folge, dass alle fünf Service-Klassen der Application-Schicht (`CompetitionService`, `PersonService`, `TicketService`, `SiteplanService`, `ContenderService`) die Spring-Annotation `@Service` tragen. Diese Annotation ist ein Framework-spezifisches Konstrukt der Plugin-Schicht und hat in einer schichtarchitektonisch sauber getrennten Application-Schicht nichts verloren. Laut Briem sind Frameworks Details, die als Plugins an den Rand der Anwendung gehören – Abhängigkeiten vom Anwendungscode in das Framework zeigen in die falsche Richtung [1, S. 58–59]. Die Application-Schicht kann aufgrund dieser Compile-Abhängigkeit nicht mehr unabhängig von Spring kompiliert oder getestet werden, was einem der Grundziele der Clean Architecture widerspricht [1, S. 16].

4

Analyse und Bewertung

Dieses Kapitel bewertet die in Kapitel 3 beschriebenen Abweichungen anhand der in Kapitel 2 eingeführten Kriterien. Für jeden der beiden Befunde wird zunächst erläutert, warum die Abweichung qualitativ problematisch ist, und anschließend werden die konkreten Auswirkungen auf die Teilmerkmale Modularity und Modifiability nach ISO/IEC 25010 bewertet.

4.1 Bewertung: Direkter Repository-Zugriff

4.1.1 Fehlende Kapselung der Anwendungslogik

Der `CompetitionController` ruft für fünf seiner sieben Endpunkte direkt Methoden des `CompetitionRepository` auf, anstatt den vorgesehenen Weg über die Application-Schicht zu nehmen. Auf den ersten Blick erscheint dies als pragmatische Abkürzung: Das Repository-Interface existiert bereits in der Domain-Schicht, es ist korrekt per Dependency Inversion entkoppelt, und die Abfragen sind einfach genug, dass ein eigener UseCase unverhältnismäßig erscheinen mag.

Diese Einschätzung greift jedoch zu kurz. Die Application-Schicht dient nicht allein als Durchleitungsebene, sondern als zentraler Ort für anwendungsspezifische Geschäftslogik [1, S. 23]. Dies zeigt sich deutlich am Vergleich der beiden Endpunkte, die den vorgesehenen Weg nutzen: Der Aufruf von `createCompetitionUseCase.create(command)` löst in `CompetitionService` eine Kette fachlicher Schritte aus — Auflösung der Team- und Official-IDs zu vollständigen Objekten, automatische Spielplanerstellung mittels `competition.scheduleMatches()` sowie anschließende Persistierung. Diese Logik wäre beim

direkten Repository-Zugriff entweder im Controller dupliziert, in das Repository verschoben oder schlicht nicht vorhanden. Das Plugin enthält damit Anwendungslogik – genau das, was Briem als grundlegenden Verstoß gegen die Rolle der Plugin-Schicht beschreibt [1, S. 37–38].

4.1.2 Auswirkung auf Modularity

Modularity nach ISO/IEC 25010 beschreibt, inwieweit Komponenten unabhängig voneinander verändert werden können [3, Abschn. 4.2.7.1]. Im vorliegenden Fall hängt der `CompetitionController` durch den direkten Import von `CompetitionRepository` und mehreren Domain-Klassen (`Competition`, `Match`, `Standings`) unmittelbar von der Domain-Schicht ab. Die Plugin- und die Domain-Schicht sind damit direkt aneinander gekoppelt, obwohl die Application-Schicht als Entkopplungsebene zwischen ihnen vorgesehen ist.

Eine Änderung an der Signatur einer Repository-Methode – etwa das Umbenennen von `findCompetitionById` oder das Anpassen des Rückgabetyps – würde damit nicht nur die Application-Schicht betreffen, sondern unmittelbar auch den Controller in der Plugin-Schicht. Die erhöhte Kopplung zwischen zwei eigentlich unabhängigen Schichten widerspricht direkt dem Ziel der Modularität, wonach Änderungen an einer Komponente möglichst geringe Auswirkungen auf andere haben sollen [3, Abschn. 4.2.7.1].

Besonders deutlich wird dies im Vergleich mit dem `PersonController`, der ausschließlich UseCase-Interfaces verwendet: Eine Änderung an `PersonRepository` hätte dort keine direkte Auswirkung auf den Controller, da dieser das Repository nicht kennt. Im `CompetitionController` ist dasselbe Schutzniveau für fünf von sieben Endpunkten nicht gegeben.

4.1.3 Auswirkung auf Modifiability

Modifiability beschreibt, ob das System effektiv verändert werden kann, ohne unbeabsichtigte Seiteneffekte zu erzeugen [3, Abschn. 4.2.7.3]. Hier treten zwei konkrete Probleme auf.

Erstens fehlt ein zentraler Ort für fachliche Erweiterungen. Soll künftig beispielsweise beim Abrufen eines Wettkampfs per `getById` eine Berechtigungsprüfung oder eine Logging-Funktion eingebaut werden, müsste dies direkt im Controller implementiert werden – oder es müsste nachträglich ein UseCase eingeführt werden, was einen Umbau des Controllers erfordert. In einem System, das die Clean Architecture konsequent umsetzt, wäre dieser Eingriff auf die Application-Schicht beschränkt; der Controller bliebe unverändert.

Zweitens besteht eine Inkonsistenz innerhalb des `CompetitionController` selbst: Zwei Endpunkte laufen korrekt über UseCase-Interfaces, fünf nicht. Diese Inkonsistenz erhöht die kognitive Last bei Weiterentwicklungen, da Entwicklerinnen und Entwickler für jeden

Endpunkt individuell prüfen müssen, welchem Muster er folgt. Ein einheitliches Muster – alle Endpunkte über UseCases – wäre einfacher zu verstehen, zu erweitern und zu testen und entspräche dem Grundsatz, dass die Plugin-Schicht ausschließlich Delegationscode enthalten soll [1, S. 37].

4.2 Bewertung: Framework-Abhängigkeit in der Application-Schicht

4.2.1 Spring als Compile-Abhängigkeit

Die Application-Schicht deklariert `spring-boot-starter-web` in ihrer `pom.xml` als Compile-Zeit-Abhängigkeit. Damit kennt und benötigt die Application-Schicht Spring, um kompiliert werden zu können. Dies ist eine strukturelle Verletzung der Dependency Rule: Abhängigkeiten sollen von außen nach innen zeigen – von der Plugin-Schicht in die Application-Schicht, nicht umgekehrt [1, S. 11]. Briem hält explizit fest, dass Frameworks Details sind, die als Plugins an den Rand der Anwendung gehören, und dass Abhängigkeiten vom Anwendungscode in das Framework in die falsche Richtung zeigen [1, S. 58–59].

Die praktische Konsequenz ist, dass die Application-Schicht nicht mehr unabhängig von Spring gebaut oder betrieben werden kann – ein Ziel, das die Clean Architecture ausdrücklich anstrebt [1, S. 16]. Zwar handelt es sich bei `@Service` um eine wenig invasive Annotation, doch die Compile-Abhängigkeit geht über die Annotation hinaus: Das gesamte `spring-boot-starter-web`-Paket steht der Application-Schicht zur Verfügung, was das Risiko erhöht, dass künftige Entwicklungen weitere Spring-spezifische Konstrukte in die Application-Schicht einbringen.

4.2.2 Auswirkung auf Modularity und Modifiability

Die Spring-Abhängigkeit in der Application-Schicht beeinträchtigt beide Qualitätskriterien. Hinsichtlich Modularity ist die Application-Schicht nicht mehr unabhängig von der Plugin-Technologie: Ein Wechsel des Web-Frameworks – der laut Clean Architecture ausschließlich die Plugin-Schicht betreffen sollte – würde nun auch die Application-Schicht berühren. Die Module sind damit stärker aneinander gebunden als die Architektur vorsieht.

Hinsichtlich Modifiability erschwert die Abhängigkeit das isolierte Testen der Application-Schicht: Komponententests der Services erfordern den Spring-Kontext oder zumindest Spring-Mocking-Infrastruktur, obwohl die fachliche Logik dieser Schicht von Spring vollständig

unabhängig sein sollte. Dies erhöht den Aufwand für Änderungen, da jede Anpassung eines Services auch unter dem Gesichtspunkt der Spring-Kompatibilität geprüft werden muss.

4.3 Risikobewertung mittels FMEA

Um die identifizierten Befunde strukturiert zu priorisieren und ihren Handlungsbedarf zu quantifizieren, wird eine Fehler-Möglichkeiten- und Einfluss-Analyse (FMEA) durchgeführt. Die FMEA ist eine etablierte Methode des Qualitätsmanagements zur systematischen Identifikation und Bewertung von Fehlerquellen [4, S. 10]. Jede Fehlerquelle wird anhand dreier Faktoren bewertet, die jeweils einen Wert zwischen 1 und 10 annehmen:

- **A** – Auftretswahrscheinlichkeit: Wie häufig bzw. mit welcher Wahrscheinlichkeit tritt der Fehler auf?
- **B** – Bedeutung der Fehlerfolge: Wie schwerwiegend sind die Auswirkungen auf das System?
- **E** – Entdeckungswahrscheinlichkeit: Wie wahrscheinlich ist es, dass der Fehler im Entwicklungsprozess entdeckt wird? (1 = zwangsläufig entdeckt, 10 = kaum entdeckbar)

Die Risikoprioritätszahl (RPZ) ergibt sich als Produkt der drei Faktoren: $RPZ = A \times B \times E$. Werte ab 100 gelten als hohes Risiko mit dringendem Handlungsbedarf [4, S. 10].

ID	Fehlerquelle	A	B	E	RPZ
F1	Direkter Repository-Zugriff im CompetitionController (5 von 7 Endpunkten umgehen die Application-Schicht)	10	7	8	560
F2	Spring-Compile-Abhängigkeit in der Application-Schicht (spring-boot-starter-web in pom.xml)	10	5	4	200

Tabelle 1 – FMEA-Analyse der identifizierten Architekturverletzungen

RPZ	Risiko	Handlungsbedarf	Maßnahmen
RPZ = 1	keins	keiner	keine
2–50	gering	nicht zwingend	können formuliert und umgesetzt werden
50–100	mittel	besteht	sollten formuliert und umgesetzt werden
100–1000	hoch	dringend	müssen formuliert und umgesetzt werden

Tabelle 2 – RPZ-Wertebereiche und Fehlerrisikoklassen (nach [4, S. 10])

Beide Fehlerquellen fallen mit einer RPZ von 560 (F1) und 200 (F2) in die Risikoklasse **hoch**, was dringendem Handlungsbedarf entspricht. Die Bewertung im Einzelnen:

F1 erhält $A = 10$, da die Verletzung bereits vollständig im Code vorhanden und damit mit Sicherheit eingetreten ist. $B = 7$ spiegelt wider, dass der direkte Repository-Zugriff die Modularity und Modifiability des Systems substanziell beeinträchtigt: Änderungen an der Domain-Schicht schlagen unmittelbar auf die Plugin-Schicht durch, und fachliche Erweiterungslogik hat keinen definierten Ort. $E = 8$ ergibt sich daraus, dass die Verletzung im normalen Entwicklungsbetrieb kaum auffällt – der Code kompiliert fehlerfrei, und funktionale Tests würden keine Abweichung zeigen. Nur eine gezielte Architekturprüfung oder ein Code-Review mit Kenntnis der Dependency Rule würde den Fehler aufdecken.

F2 erhält ebenfalls $A = 10$, da die Spring-Abhängigkeit in der `pom.xml` strukturell verankert ist. $B = 5$ reflektiert, dass die unmittelbaren funktionalen Auswirkungen begrenzter sind als bei F1 – das System läuft korrekt, solange Spring verwendet wird. Die Beeinträchtigung betrifft vor allem die langfristige Austauschbarkeit des Frameworks und die Testbarkeit der Application-Schicht ohne Spring-Kontext. $E = 4$ ist niedriger als bei F1, da die Abhängigkeit in der `pom.xml` explizit sichtbar ist und bei einem gezielten Blick auf die Modulkonfiguration auffällt.

4.4 Gesamtbewertung

Beide Befunde sind keine isolierten Einzelfehler, sondern Ausdruck desselben Grundproblems: Die Dependency Rule wurde an der Grenze zwischen Plugin-Schicht und Application-Schicht nicht konsequent eingehalten. Im Kern der Anwendung – Domain- und Adapter-Schicht –

ist die Architektur vorbildlich umgesetzt. An der äußersten Grenze, wo die technische Infrastruktur auf die fachliche Logik trifft, entstehen jedoch Kurzschlussverbindungen, die die mit der Clean Architecture angestrebten Qualitätsziele Modularity und Modifiability untergraben.

Die FMEA-Analyse bestätigt diesen Befund quantitativ: Beide Fehlerquellen liegen mit RPZ-Werten von 560 und 200 deutlich im Bereich hohen Risikos und erfordern damit nach [4, S. 10] das Formulieren und Umsetzen konkreter Maßnahmen. F1 ist dabei als dringlicher einzustufen, da die Auswirkungen auf Modularity und Modifiability breiter und schwerer zu kontrollieren sind. Die entsprechenden Optimierungsmaßnahmen werden in Kapitel 5 erarbeitet.

5

Optimierungsmaßnahmen

5.1 Repository-Zugriffe in Controller eliminieren

5.2 Service-Annotation verschieben

5.3 Erwartete Auswirkungen im PDCA-Rahmen

6

Fazit

A Literatur

- [1] L. Briem, „Clean Architecture: Systeme für die Ewigkeit“, 2024.
- [2] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Pearson Education, 2018.
- [3] ISO/IEC, „Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — System and software quality models“, Nr. ISO/IEC25010:2011. 2011.
- [4] D. Kube und J. Schwarzenböck, „Softwarequalität“, 2023.

B Glossar

Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden- Württemberg vom 29.09.2017. Ich versichere hiermit, dass ich meine Arbeit mit dem Thema:

Softwarequalität

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass alle eingereichten Fassungen übereinstimmen.

Karlsruhe, 19.05.2026

Etienne Luke Josef Bader